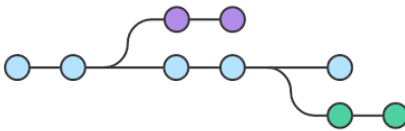


Git Prune: Check Out this Housekeeping Utility

We carry out a commit after every small change we make in the source code to ensure that all the changes are reserved. However, when we make the final push to master the code, there may be several commits we need to ignore or detach. We often have unreachable or orphaned Git objects, and they need to be cleaned up. Git Prune helps with that.



As we know, git is very careful about deleting data, and does not lose any commits easily. The downside to this is that lots of stale data is reserved even if it's not going to be used. One way to clean this data is by using *prune*.

Git prune

The *git prune* command is a housekeeping utility that is used to clean up unreachable or orphaned git objects. Unreachable objects are those that are not accessible by any refs. As an example, say you have made two commits, and reset them to different heads by calling the *git reset <commit id>* command. Though the *git log* command will not show the commit info, git still stores it internally as a dangling object.

Users need not call the *git prune* command directly, but can call *git gc*, the git garbage collection command. This will prune the data along with many other housekeeping tasks.

Usage

The following command:

```
git prune --dry-run
```

...will not carry out the prune operation. Instead, it will indicate what the output will be if this operation is performed.

The command:

```
git prune --verbose
```

...will report all the actions and its associated objects.

The command given below:

```
git prune --progress
```

...will show the progress made by *git prune*.

The following command:

```
git prune --expire <time>
```

...will remove only the objects that are older than the stipulated time.

We will use a simple example to see how this works. To begin with, we will clone one of the git repositories or even create your own repository using *git init*. I will be cloning a repository for explaining the usage of the *prune* command:

```
% git clone https://github.com/hasura/graphql-engine.git
```

```
Cloning into 'graphql-engine'...
```

```
remote: Enumerating objects: 66417, done.
```

```
remote: Counting objects: 100% (5997/5997), done.
```

```
remote: Compressing objects: 100% (2118/2118), done.
```

```
remote: Total 66417 (delta 3840), reused 5775 (delta 3718),  
pack-reused 60420
```

```
Receiving objects: 100% (66417/66417), 77.20 MiB | 3.11  
MiB/s, done.
```

```
Resolving deltas: 100% (44018/44018), done.
```

```
Updating files: 100% (5104/5104), done.
```

We will now create a sample file with the text inserted in the first line. You can create this file in any text editor of your choice.

```
% cat >> hello.txt
```

This is the first line

Add the text file and commit:

```
% git add hello.txt
```